

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



KIỂM TRA CUỐI KỲ

NHẬP MÔN HỌC MÁY

—o0o—

TÁC GIẢ

22127180 Nguyễn Phúc Khang

LỚP

22KHMT1

—o0o—

GIẢNG VIÊN HƯỚNG DẪN

Bùi Tiến Lên

Lê Nhật Nam

Võ Nhật Tân

Nguyễn Thanh Tình

THÀNH PHỐ HỒ CHÍ MINH, 8 2025

MỤC LỤC

MỤC LỤC	i
LIST OF TABLES	ii
1 Giới thiệu	1
1.1 Mô tả bài toán	1
1.2 Phân chia dữ liệu	1
1.3 Mục tiêu của báo cáo	1
2 Thư viện sử dụng	2
3 Tiền xử lý dữ liệu	3
3.1 Kiểm tra và làm sạch dữ liệu	3
3.2 Phân tích tương quan và chọn đặc trưng	3
3.3 Chuẩn hóa dữ liệu	5
4 Huấn luyện và chọn mô hình	6
4.1 K-Fold Cross Validation	6
4.2 Mô hình 1: K-Nearest Neighbors (KNN)	6
4.2.1 Lý do chọn mô hình	6
4.2.2 Các hàm trong Class KNN	6
4.2.3 Cách hoạt động của mô hình KNN	7
4.2.4 Kết quả huấn luyện và kiểm tra	7
4.2.5 Nhận xét kết quả	8
4.3 Mô hình 2: Neural Network	8
4.3.1 Lý do chọn mô hình	8
4.3.2 Các hàm trong Class Neural Network	8
4.3.3 Cách hoạt động của mô hình Neural Network	9
4.3.4 Kết quả huấn luyện và kiểm tra	9
4.3.5 Nhận xét kết quả	10
4.3.6 Các Khám Phá Khác	10
5 Tổng Kết	11
5.1 So sánh kết quả	11
5.2 Giải quyết nhược điểm	11

LIST OF TABLES

3.1	Thông tin các cột trong tập huấn luyện (<code>train.csv</code>).	3
3.2	Hệ số tương quan giữa các đặc trưng và nhãn <code>price_range</code>	4
4.1	Kết quả KFold Cross-Validation	7
4.2	Kết quả trên tập huấn luyện và kiểm tra	7
4.3	Kết quả KFold Cross-Validation	9
4.4	Kết quả trên tập huấn luyện và kiểm tra.	10

Giới thiệu

1.1 Mô tả bài toán

Nhiệm vụ chính của bài toán này là dựa trên đặc trưng của các dòng điện thoại có thể dự đoán được khoảng giá bán (`price_range`) của dòng điện thoại đó

1.2 Phân chia dữ liệu

Dữ liệu được chia thành hai tập chính:

- **Tập huấn luyện (`train.csv`):** Gồm 2000 mẫu, dùng để huấn luyện mô hình.
- **Tập kiểm tra (`test.csv`):** Gồm 1000 mẫu, dùng để đánh giá hiệu suất mô hình.

1.3 Mục tiêu của báo cáo

Báo cáo sử dụng hai mô hình học máy sau đây để giải quyết bài toán:

- **K-Nearest Neighbors (KNN):** Mô hình dựa trên khoảng cách để dự đoán nhãn bằng cách tham chiếu đến k điểm gần nhất.
- **Neural Network (NN):** Mô hình dựa trên các lớp nơ-ron phi tuyến, học hàm ánh xạ từ đặc trưng đầu vào đến nhãn đầu ra.

Thư viện sử dụng

- **NumPy**: Thư viện cơ bản cho các phép toán số học và đại số tuyến tính. Được sử dụng để tính toán khoảng cách Euclidean trong KNN và các phép toán trong Neural Network.
- **Pandas**: Thư viện cho việc quản lý và thao tác dữ liệu với cấu trúc DataFrame. Sử dụng để tải và chuẩn bị dữ liệu.
- **Seaborn**: Thư viện trực quan hóa dữ liệu dựa trên Matplotlib, được sử dụng để vẽ biểu đồ phân phối và ma trận tương quan.
- **Matplotlib**: Thư viện vẽ đồ thị, giúp trực quan hóa kết quả huấn luyện và các biểu đồ khác.
- **Scikit-learn**: Thư viện học máy, sử dụng `MinMaxScaler` để chuẩn hóa dữ liệu và `KFold` để thực hiện K-Fold Cross Validation.

Tiền xử lý dữ liệu

3.1 Kiểm tra và làm sạch dữ liệu

Dữ liệu được kiểm tra qua lệnh `train_data.info()` với kết quả như Bảng 3.1.

Cột dữ liệu	Số lượng không null	Kiểu dữ liệu
id	2000	int64
battery_power	2000	int64
blue	2000	int64
clock_speed	2000	float64
dual_sim	2000	int64
fc	2000	int64
four_g	2000	int64
int_memory	2000	int64
m_dep	2000	float64
mobile_wt	2000	int64
n_cores	2000	int64
pc	2000	int64
px_height	2000	int64
px_width	2000	int64
ram	2000	int64
sc_h	2000	int64
sc_w	2000	int64
talk_time	2000	int64
three_g	2000	int64
touch_screen	2000	int64
wifi	2000	int64
price_range	2000	int64

Table 3.1: Thông tin các cột trong tập huấn luyện (`train.csv`).

Nhận xét:

- Không có giá trị thiếu (`null`).
- Cột `id` không hữu ích cho mô hình, ta sẽ được loại bỏ cột dữ liệu này.

3.2 Phân tích tương quan và chọn đặc trưng

Dựa trên hệ số tương quan và biểu đồ ma trận tương quan (Hình 3.1), các đặc trưng có mối tương quan lớn hơn 0 `price_range` được chọn để huấn luyện mô hình.

Đặc trưng	Hệ số tương quan
ram	0.917046
battery_power	0.200723
px_width	0.165818
px_height	0.148858
int_memory	0.044435
sc_w	0.038711
pc	0.033599
three_g	0.023611
sc_h	0.022986
fc	0.021998
talk_time	0.021859
blue	0.020573
wifi	0.018785
dual_sim	0.017444
four_g	0.014772
n_cores	0.004399
m_dep	0.000853

Table 3.2: Hệ số tương quan giữa các đặc trưng và nhãn price_range.

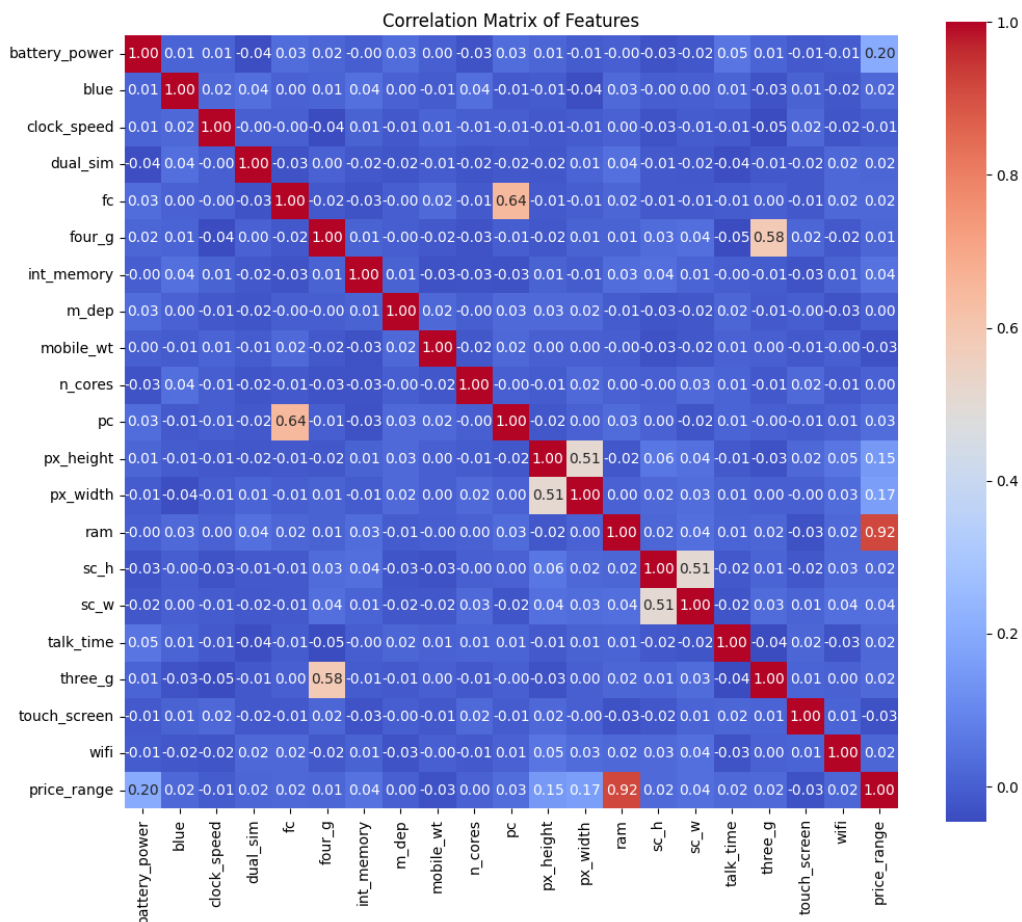


Figure 3.1: Ma trận tương quan giữa các đặc trưng trong tập dữ liệu.

3.3 Chuẩn hóa dữ liệu

Để cải thiện hiệu suất và tốc độ huấn luyện của mô hình, dữ liệu được chuẩn hóa bằng hai phương pháp chính:

- **StandardScaler:** Chuẩn hóa dữ liệu theo phân phối chuẩn với `mean=0` và `std=1`, giúp dữ liệu có giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1.
- **MinMaxScaler:** Dữ liệu được đưa về khoảng giá trị $[0, 1]$, đảm bảo tất cả các đặc trưng có thang đo đồng nhất, từ đó giúp mô hình học nhanh và hiệu quả hơn.

Nhận xét về từng mô hình:

- **KNN:** Sau khi thử nghiệm, MinMaxScaler mang lại kết quả tốt hơn so với StandardScaler. Điều này là do KNN là một mô hình dựa trên khoảng cách giữa các điểm, và nó rất nhạy cảm với thang đo của dữ liệu. Khi dữ liệu có giá trị trên các phạm vi khác nhau, KNN có thể ưu tiên các đặc trưng có giá trị lớn, làm giảm độ chính xác của mô hình.
- **Neural Network:** Với Neural Network, việc sử dụng MinMaxScaler cũng giúp cải thiện tốc độ hội tụ của mô hình. Đặc biệt với hàm kích hoạt ReLU, việc đưa dữ liệu vào khoảng $[0, 1]$ giúp giảm khả năng xảy ra hiện tượng *vanishing gradient*, từ đó tăng tốc độ huấn luyện và giúp mô hình hội tụ nhanh hơn.

Giải thích thêm về hiện tượng *vanishing gradient*: Trong quá trình huấn luyện mạng neural, đặc biệt khi sử dụng hàm kích hoạt như Sigmoid hoặc Tanh, gradient có thể trở nên rất nhỏ khi truyền qua nhiều lớp. Điều này dẫn đến việc các trọng số trong các lớp sâu không được cập nhật hiệu quả, làm cho mô hình không thể học được các đặc trưng quan trọng. Với hàm kích hoạt ReLU, khi đầu vào có giá trị âm, gradient sẽ bằng 0, còn khi đầu vào dương, gradient sẽ giữ nguyên, giúp giảm thiểu hiện tượng *vanishing gradient*. Khi dữ liệu được chuẩn hóa vào khoảng $[0, 1]$, khả năng các giá trị đầu vào của ReLU lớn hơn 0 cũng cao hơn, từ đó giảm thiểu nguy cơ này và cải thiện tốc độ hội tụ của mô hình.

Huấn luyện và chọn mô hình

4.1 K-Fold Cross Validation

Trước khi huấn luyện các mô hình, ta sử dụng kỹ thuật **K-Fold Cross Validation** để đánh giá và tối ưu mô hình tốt hơn. Kỹ thuật này chia tập huấn luyện thành K phần (folds) và tiến hành huấn luyện mô hình trên $K - 1$ phần dữ liệu, kiểm tra trên phần còn lại. Quá trình này sẽ được lặp lại K lần, mỗi lần với một phần dữ liệu khác nhau làm tập kiểm tra.

Việc sử dụng K-Fold giúp giảm thiểu overfitting và cung cấp một cách đánh giá ổn định hơn so với việc chia dữ liệu thành một tập huấn luyện và một tập kiểm tra duy nhất. Sau khi kết thúc các vòng lặp, ta sẽ tính trung bình độ chính xác từ tất cả các lần kiểm tra để đánh giá tổng thể hiệu quả của từng mô hình.

- **K=5:** Việc sử dụng 5-fold cross-validation để đánh giá mô hình vì đây là giá trị thường được sử dụng để đảm bảo tính ổn định mà không gây tốn quá nhiều tài nguyên tính toán.
- **Ưu điểm:** K-Fold giúp giảm thiểu sự phụ thuộc vào cách phân chia dữ liệu và làm cho kết quả đánh giá mô hình trở nên đáng tin cậy hơn.

Sau khi hoàn thành bước cross-validation, chúng tôi sẽ chuyển sang huấn luyện các mô hình KNN và Neural Network.

4.2 Mô hình 1: K-Nearest Neighbors (KNN)

4.2.1 Lý do chọn mô hình

KNN dự đoán giá bán bằng cách tìm k điểm gần nhất và bỏ phiếu đa số. Đây là mô hình đơn giản, dễ triển khai và không yêu cầu nhiều giả định về dữ liệu.

4.2.2 Các hàm trong Class KNN

- `__init__`:
 - Hàm khởi tạo xác định số lượng các điểm gần (k) để tìm dựa trên tham số truyền vào. Mặc định là $k = 5$.
- `load_model_weights`:
 - Do KNN là một mô hình không tham số (non-parametric), nên không có trọng số để load như các mô hình khác, chỉ in ra thông báo khi gọi.

- **train:**
 - Hàm huấn luyện mô hình KNN bằng cách lưu trữ tập huấn luyện (**X_train**) và nhãn (**y_train**) để sử dụng trong quá trình dự đoán.
- **predict:**
 - Hàm nhận đầu vào là một tập các điểm dữ liệu và trả về dự đoán cho mỗi điểm bằng cách gọi hàm **_predict**.
- **_predict:**
 - Hàm tính toán khoảng cách Euclidean giữa điểm cần dự đoán và tất cả các điểm trong tập huấn luyện, sau đó chọn k điểm gần nhất để bỏ phiếu và đưa ra dự đoán cho nhãn.
- **evaluate:**
 - Hàm tính toán độ chính xác và loss của mô hình trên một tập kiểm tra (**X_test** và **y_test**).

4.2.3 Cách hoạt động của mô hình KNN

Mô hình KNN (K-Nearest Neighbors) hoạt động bằng cách tính khoảng cách Euclidean giữa điểm cần dự đoán và tất cả các điểm trong tập huấn luyện. Sau đó, mô hình sẽ chọn ra $k = 5$ điểm gần nhất và quyết định kết quả dựa trên bỏ phiếu đa số của các điểm này.

4.2.4 Kết quả huấn luyện và kiểm tra

- **Kết quả KFold Cross-Validation:**

Fold	Accuracy	Loss
Fold 1	0.4500	0.5500
Fold 2	0.4350	0.5650
Fold 3	0.4975	0.5025
Fold 4	0.4125	0.5875
Fold 5	0.4050	0.5950

Table 4.1: Kết quả KFold Cross-Validation

- **Kết quả trên tập huấn luyện và kiểm tra:**

Tập Dữ Liệu	Accuracy	Loss
Tập Huấn Luyện	0.6740	0.3260
Tập Kiểm Tra	0.2510	0.7490

Table 4.2: Kết quả trên tập huấn luyện và kiểm tra

4.2.5 Nhận xét kết quả

- **Kết quả Cross-Validation:** Độ chính xác trung bình đạt 44% và độ mất mát trung bình là 56%. Mô hình có sự dao động giữa các fold.
- **Kết quả Tập Huấn Luyện:** Độ chính xác đạt 67.4%, cho thấy mô hình đã học khá tốt từ dữ liệu huấn luyện.
- **Kết quả Tập Kiểm Tra:** Độ chính xác trên tập kiểm tra chỉ đạt 25.1%, cho thấy mô hình có khả năng bị hiện tượng overfitting, không tổng quát tốt cho dữ liệu mới.

4.3 Mô hình 2: Neural Network

4.3.1 Lý do chọn mô hình

Mô hình Neural Network được chọn vì khả năng học được các đặc trưng phức tạp từ dữ liệu không tuyến tính, khả năng tổng quát cao và dễ dàng điều chỉnh kiến trúc mạng để cải thiện hiệu suất. Mạng nơ-ron có thể xử lý các bài toán phân loại đa lớp, phù hợp với đặc tính của bài toán này.

4.3.2 Các hàm trong Class Neural Network

- `__init__`:
 - Hàm khởi tạo với các tham số: số lượng epoch (`epochs`), tốc độ học (`learning_rate`), kích thước lớp đầu vào (`input_size`), lớp ẩn và lớp đầu ra. Các trọng số được khởi tạo ngẫu nhiên.
- `save_model_weights`:
 - Hàm lưu trọng số và bias của mô hình vào file `weight.json`.
- `load_model_weights`:
 - Hàm đọc file `weight.json` để khôi phục trọng số và bias vào mô hình.
- `one_hot_encode`:
 - Hàm chuyển đổi nhãn đầu ra thành dạng one-hot encoding.
- `relu`:
 - Hàm kích hoạt ReLU (Rectified Linear Unit) được sử dụng trong các lớp ẩn.
- `softmax`:
 - Hàm softmax được sử dụng ở lớp đầu ra để tính xác suất cho các lớp.
- `forward_pass`:

- Hàm thực hiện bước lan truyền tiến, tính toán giá trị đầu ra của các lớp trong mạng.
- `compute_loss`:
 - Hàm tính toán mất mát (loss) giữa dự đoán và nhãn thật của tập huấn luyện.
- `backprop`:
 - Hàm thực hiện bước lan truyền ngược (backpropagation) để tính toán các gradient của trọng số và bias trong mạng.
- `update_weights`:
 - Hàm cập nhật trọng số và bias của mô hình dựa trên các gradient tính được.
- `train`:
 - Hàm huấn luyện mô hình Neural Network với số lượng epoch đã chỉ định và tốc độ học.
- `evaluate`:
 - Hàm tính toán độ chính xác của mô hình trên tập kiểm tra.
- `loss`:
 - Hàm tính toán lỗi (loss).

4.3.3 Cách hoạt động của mô hình Neural Network

Mô hình Neural Network hoạt động theo cơ chế truyền qua các lớp (forward pass) và cập nhật trọng số dựa trên gradient của các lỗi dự đoán (backpropagation). Các lớp ẩn sử dụng hàm kích hoạt ReLU để chuyển đổi đầu vào thành một không gian phi tuyến, lớp đầu ra sử dụng hàm softmax để tính xác suất cho các lớp phân loại. Mô hình sử dụng thuật toán Gradient Descent để tối ưu hóa trọng số.

4.3.4 Kết quả huấn luyện và kiểm tra

- **Kết quả KFold Cross-Validation:**

Fold	Accuracy	Loss
Fold 1	0.2250	1.3878
Fold 2	0.2625	1.3847
Fold 3	0.3375	1.3819
Fold 4	0.2900	1.3820
Fold 5	0.3225	1.3825

Table 4.3: Kết quả KFold Cross-Validation

- **Kết quả tổng hợp trên tập huấn luyện và kiểm tra:**

Tập dữ liệu	Độ chính xác (%)	Độ mất mát
Tập huấn luyện	0.3710	1.3805
Tập kiểm tra	0.3220	1.3819

Table 4.4: Kết quả trên tập huấn luyện và kiểm tra.

4.3.5 Nhận xét kết quả

- **Kết quả Cross-Validation:** Độ chính xác trung bình đạt 28.75% và độ mất mát trung bình là 1.3838. Mô hình có sự dao động giữa các fold.
- **Kết quả Tập Huấn Luyện:** Độ chính xác đạt 37.10% và độ mất mát là 1.3805, cho thấy mô hình đã học từ dữ liệu huấn luyện nhưng hiệu suất chưa cao.
- **Kết quả Tập Kiểm Tra:** Độ chính xác trên tập kiểm tra đạt 32.20% và độ mất mát là 1.3819, cao hơn một chút so với tập huấn luyện nhưng vẫn còn hạn chế, chưa đủ tổng quát cho dữ liệu mới.

4.3.6 Các Khám Phá Khác

Trong quá trình thử nghiệm, cấu trúc các tầng (layer) của mô hình Neural Network đã được điều chỉnh để tìm ra giá trị mang lại độ chính xác cao nhất. Qua các thử nghiệm, cấu trúc với số lượng node ở các tầng được thiết lập như sau:

- **Tầng đầu vào:** Kích thước bằng số đặc trưng của tập dữ liệu.
- **Tầng ẩn thứ nhất:** Số lượng node gấp đôi kích thước đầu vào.
- **Tầng ẩn thứ hai:** Số lượng node bằng kích thước đầu vào.
- **Tầng đầu ra:** Số lượng node bằng số lớp cần phân loại.

Cấu trúc này mang lại độ chính xác cao nhất trong các thử nghiệm. Tầng ẩn đầu tiên với số lượng node gấp đôi kích thước đầu vào giúp mô hình có khả năng học tốt hơn các biểu diễn phức tạp từ dữ liệu, trong khi tầng ẩn thứ hai duy trì kích thước bằng kích thước đầu vào để giảm thiểu độ phức tạp, giúp tránh overfitting. Tầng đầu ra phù hợp với số lớp của bài toán phân loại, đảm bảo tính chính xác trong dự đoán.

Khám phá này cho thấy việc lựa chọn số lượng node trong các tầng ẩn có thể ảnh hưởng lớn đến hiệu suất của mô hình Neural Network và cần được điều chỉnh cẩn thận trong từng bài toán cụ thể.

Tổng Kết

5.1 So sánh kết quả

- Kết quả của cả hai mô hình đều có độ chính xác trên tập kiểm tra khá thấp. Cụ thể, độ chính xác của mô hình KNN là khoảng 25.10%, trong khi mô hình Neural Network đạt độ chính xác khoảng 32.70%. Mặc dù Neural Network có độ chính xác cao hơn một chút, cả hai mô hình đều không thể đạt được kết quả tốt, cho thấy mô hình chưa đủ mạnh.
- Một yếu tố quan trọng ảnh hưởng đến kết quả là kích thước bộ dữ liệu huấn luyện và kiểm tra. Cả hai bộ dữ liệu này đều khá nhỏ, dẫn đến việc mô hình gặp khó khăn trong việc học các đặc trưng và dễ dàng gặp phải hiện tượng overfitting hoặc underfitting. Việc thiếu dữ liệu đã làm giảm khả năng tổng quát của mô hình, khiến các kết quả đánh giá chưa thực sự phản ánh hiệu suất tối ưu của mô hình.

5.2 Giải quyết nhược điểm

- Thu thập thêm dữ liệu huấn luyện để cải thiện tính tổng quát của mô hình.
- Đảm bảo rằng tập huấn luyện và kiểm tra có phân bố đồng đều các lớp.
- Tạo thêm dữ liệu từ các ví dụ hiện có để cải thiện khả năng học của mô hình.
- Thử nghiệm với các kiến trúc mạng khác nhau, thay đổi learning rate hoặc thêm dropout để cải thiện hiệu suất.